

Minuit

BOUTIQUE HALLOWEEN

Conception et réalisation de la boutique

Minuit sur AWS

Rapport de projet : Infrastructures cloud et virtualisation

IMT Nord Europe

FISA TI 2027

Bahir Boudouma Lambarki

Tarik Fellahi

Matys Lepretre

Enseignant : Raouf Boulkroune

Résumé

Ce projet étudie la conception d'une boutique saisonnière capable d'enregistrer des commandes, de confirmer leur réception et de déclencher des traitements asynchrones. Notre réalisation combine une API FastAPI sur EC2, une base PostgreSQL et des fonctions Lambda alimentées par SNS et SQS. Terraform décrit l'infrastructure et Docker Compose fournit un environnement local reproductible.

Nous comparons cette solution hybride à une cible d'industrialisation intégrant davantage de disponibilité, d'orchestration et de capacités de reprise. La réalisation permet d'étudier les transactions, l'idempotence et les contraintes d'exploitation AWS. Elle ne constitue pas une plateforme commerciale validée en haute disponibilité : la base du profil livré reste Single-AZ et les performances en charge n'ont pas été mesurées. Le coût permanent du réseau et de la base montre également que l'usage de fonctions serverless ne suffit pas à garantir un fonctionnement gratuit.

Mots clés : AWS, Terraform, EC2, Lambda, architecture événementielle, idempotence, disponibilité.

Sommaire

TOC \f M \l "1-2" \h \z * MERGEFORMAT 1 Contexte et démarche de conception 3
1.1 Problématique et objectifs 3
1.2 Comparaison des modes d'exécution 3
1.3 Choix et démarche 3
2 Architecture hybride et cohérence des commandes 4
2.1 Organisation des services 4
2.2 Parcours d'une commande 4
2.3 Fiabilité et limites transactionnelles 4
3 Cible d'industrialisation et analyse économique 6
3.1 Évolution proposée 6
3.2 Reprise régionale 6
3.3 Comparaison économique 6
4 Réalisation et reproductibilité 8
4.1 Organisation technique 8
4.2 Installation locale 8
4.3 Déploiement et sécurité 8
4.4 Déroulement et décisions du projet 9
4.5 Reproduire le déploiement Terraform 9
5 Validation et résultats 10
5.1 Méthode 10
5.2 Contrôles du 30 octobre 2026 10
5.3 Essais AWS conservés 10
5.4 Limites des résultats 10
5.5 Scénario de démonstration pour l'évaluation 11
5.6 Vérifier une nouvelle tentative et les accès 11
5.7 Relancer les contrôles automatisés 11
6 Discussion et apprentissages 12
6.1 Cohérence métier et systèmes distribués 12
6.2 Écarts entre local et cloud 12
6.3 Disponibilité et proportionnalité 12
6.4 Perspectives hiérarchisées 12
7 Conclusion et références 13
7.1 Conclusion 13
7.2 Références techniques 13

1 Contexte et démarche de conception

1.1 Problématique et objectifs

Le sujet pédagogique demande de réaliser une boutique en ligne associant consultation d'un catalogue, prise de commande et confirmation au client. L'entreprise doit conserver les commandes pour l'audit et déclencher des opérations internes d'emballage et d'expédition simulée. Les contraintes portent sur l'évolutivité, la résilience, le coût et l'automatisation de l'infrastructure [1].

Nous avons choisi une boutique Halloween nommée Minuit. Ce contexte saisonnier met en évidence une difficulté classique : la capacité nécessaire pendant une pointe peut être largement supérieure à celle utilisée le reste de l'année. Notre problématique est donc de concilier cohérence des commandes, adaptation de la capacité et maîtrise du coût permanent.

Le périmètre fonctionnel comprend douze produits de démonstration, un panier, des comptes clients, un historique et deux notifications email. Le paiement, l'emballage physique et le transport sont exclus. La confirmation affichée après la commande signifie que son enregistrement a réussi ; les opérations logistiques et les emails sont ensuite asynchrones.

1.2 Comparaison des modes d'exécution

Le sujet invite à comparer une solution sur machines virtuelles et une solution serverless. Une API sur EC2 permet de conserver des processus persistants et un modèle Python proche du développement local. Elle impose en contrepartie de gérer les instances, les mises à jour et une capacité minimale.

Une variante entièrement serverless pourrait utiliser API Gateway et Lambda pour l'API, avec DynamoDB ou une base SQL dont les connexions seraient contrôlées. Cette variante demanderait d'adapter les points d'entrée, les accès aux données et les tests. Elle est une alternative de conception, non une deuxième implémentation livrée. Son coût devrait être évalué avec les volumes et les services associés.

Critère	Machines virtuelles	Serverless
Exécution	Processus persistants sur EC2	Fonctions invoquées à la demande
Capacité	Minimum permanent et Auto Scaling	Concurrence et quotas à maîtriser
Exploitation	Système, déploiement et instances	Configuration, événements et reprises
Intérêt dans le projet	API et publication de l'outbox	Traitements métier indépendants

1.3 Choix et démarche

Nous retenons une combinaison : EC2 pour l'API et le publisher, Lambda pour les consommateurs d'événements. Cette séparation garde les transactions SQL explicites et découple les traitements de la réponse HTTP. SNS/SQS répond au besoin de distribution et de mise en attente ; aucun besoin établi de streaming Kafka ne justifie l'ajout de MSK.

La démarche distingue un socle local, une infrastructure hybride reproductible et une cible industrielle étudiée séparément. Les preuves conservées retracent les tests locaux du 28 octobre, les essais AWS du 29 octobre et les contrôles du dépôt du 30 octobre. La cible industrialisée demeure hybride : elle ne remplace pas l'alternative entièrement serverless demandée dans la comparaison initiale.

2 Architecture hybride et cohérence des commandes

2.1 Organisation des services

CloudFront constitue le point d'entrée du navigateur. Il distribue les fichiers du frontend depuis S3 privé et transmet les appels API à un ALB interne au moyen d'une origine VPC. L'ALB dessert les instances EC2 exécutant FastAPI. Cognito assure l'authentification cloud et RDS PostgreSQL conserve les données métier. Le schéma détaillé est intégré à la planche 1 qui suit ce chapitre [2].

Le navigateur communique en HTTPS. Les liens privés CloudFront vers ALB puis ALB vers EC2 utilisent HTTP ; les connexions PostgreSQL utilisent TLS avec vérification du certificat. Les EC2 et RDS n'ont pas d'adresse IP publique. Systems Manager remplace l'accès SSH dans cette architecture.

Le profil réduit utilise une EC2 désirée, un NAT et une base RDS db.t4g.micro Single-AZ de 20 Gio. L'option high_availability augmente le minimum EC2 et prévoit un NAT par zone, mais conserve cette base Single-AZ et un jour de sauvegardes. Son nom ne doit donc pas être interprété comme une garantie de haute disponibilité de bout en bout.

2.2 Parcours d'une commande

1. Le navigateur transmet les identifiants des produits, leurs quantités et une clé d'idempotence. Il ne décide pas des prix.
2. L'API vérifie l'identité, recalcule le montant en centimes et contrôle le stock.
3. Une transaction enregistre la commande, ses lignes, l'événement métier et une entrée d'outbox. Un échec annule l'ensemble des écritures concernées.
4. Après le commit, le client reçoit sa confirmation. Le publisher lit ensuite l'outbox et publie l'événement dans SNS.
5. SNS alimente les files SQS des consommateurs. Les Lambdas réalisent l'emballage et l'expédition simulés, puis enregistrent les événements de progression.
6. Le consommateur de notification traite CommandeCreee et ExpeditionTerminee afin d'envoyer les deux emails avec SES.

2.3 Fiabilité et limites transactionnelles

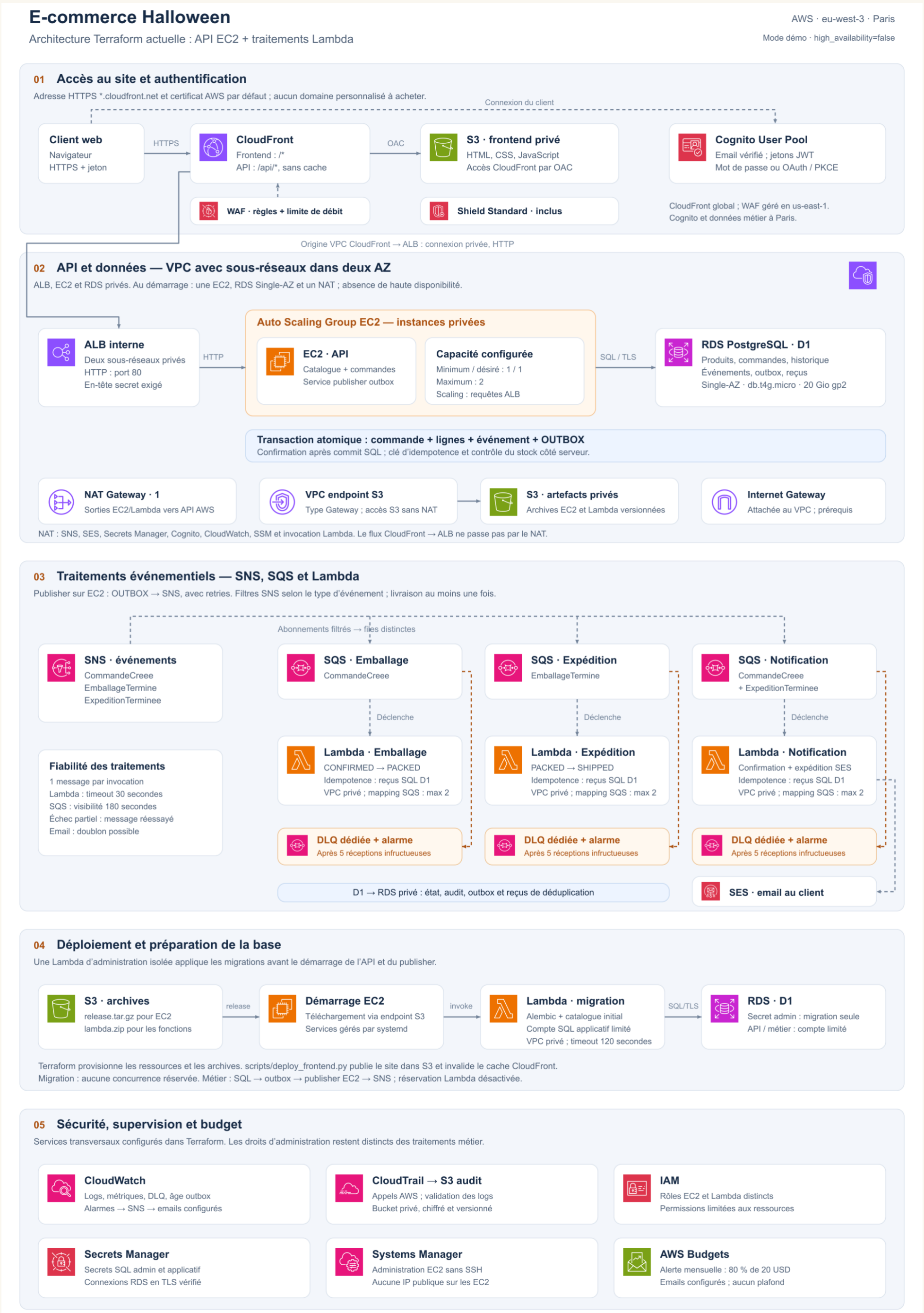
L'outbox traite le risque de double écriture entre la base et le service de messagerie : la commande et l'événement à publier sont enregistrés ensemble. Si SNS est temporairement indisponible, la publication peut être réessayée sans perdre cette intention [4].

Une répétition reste possible après une publication réussie suivie d'une interruption. Les consommateurs doivent donc accepter de recevoir plusieurs fois le même événement. Les reçus de traitement et les contraintes SQL protègent les transitions métier ; la clé d'idempotence protège également les nouvelles tentatives du client. Un panier différent présenté avec une clé déjà utilisée est refusé.

Cette protection n'offre pas une garantie générale d'exécution exactement une fois. Un arrêt après acceptation d'un email par SES et avant l'écriture du reçu SQL peut conduire à un nouvel envoi. Un futur transporteur réel devrait disposer d'un mécanisme d'idempotence propre. L'intégration Lambda/SQS peut livrer des messages plusieurs fois et demande des traitements adaptés ainsi que des réponses partielles aux lots en erreur [5].

Les files d'échec et les procédures de rejeu servent à traiter les erreurs persistantes. Elles ne remplacent pas la surveillance de leur contenu ni la recherche de la cause avant une relance.

Planche 1 Architecture hybride réalisée



Source : schéma du projet.

3 Cible d'industrialisation et analyse économique

3.1 Évolution proposée

La seconde architecture documentée constitue une cible pour une activité plus exigeante. Elle conserve EC2 pour l'API, ajoute des publishers dédiés, remplace RDS par Aurora PostgreSQL avec RDS Proxy et utilise Valkey pour les lectures fréquentes du catalogue. Le prix est néanmoins revérifié dans la base transactionnelle lors de la commande : le cache ne doit pas devenir l'autorité métier [3].

Step Functions orchestre les opérations et leurs reprises. L'archivage S3, Glue et Athena sépare les besoins analytiques des transactions. La cible prévoit également des déploiements progressifs, une supervision enrichie et une séparation des environnements. Ces composants représentent des perspectives de conception et ne sont pas créés par le Terraform livré.

3.2 Reprise régionale

La production à Paris serait complétée par un warm standby en Irlande : un environnement réduit serait maintenu prêt à être activé. Aurora Global Database assurerait la réplication des données selon la conception retenue. Les traitements à effets métier resteraient désactivés dans la région passive.

La bascule demanderait d'isoler l'ancienne région, de promouvoir la base de secours, de vérifier les accès, de modifier le routage, puis de réconcilier les commandes et les traitements. La procédure doit éviter deux régions actives simultanément. Le failover standard d'un groupe d'origines CloudFront ne couvre pas les requêtes POST ; il ne suffit donc pas à assurer la reprise des commandes [6].

Les files SQS et les exécutions en cours nécessitent une stratégie de reconstruction depuis les données durables. Les objectifs de durée de reprise (RTO) et de perte de données acceptable (RPO) doivent être fixés puis mesurés. Aucune continuité sans interruption n'est démontrée.

3.3 Comparaison économique

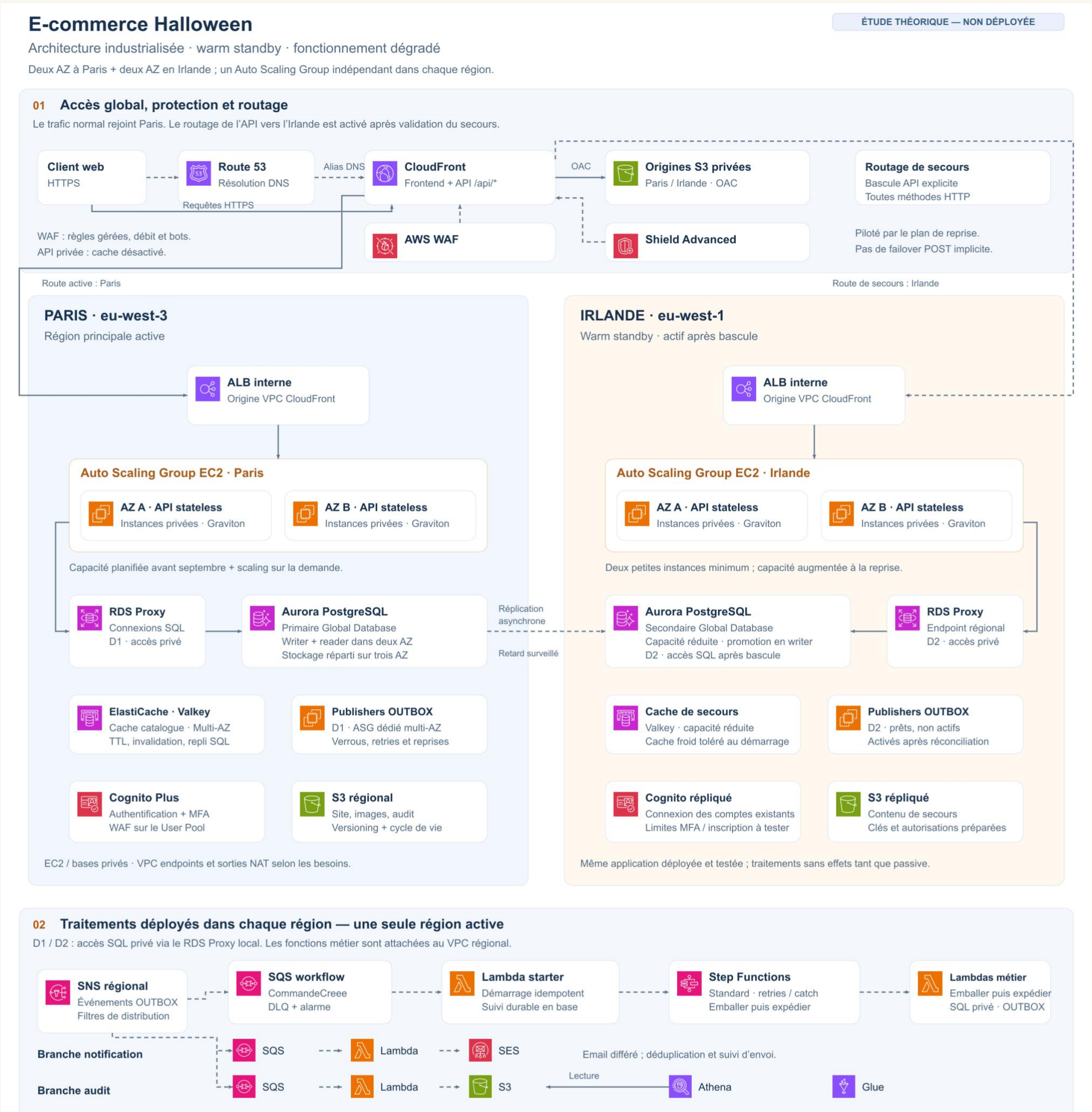
Les estimations conservées dans le dépôt datent des 29 et 30 octobre 2026. Leurs montants sont hors taxes, fondés sur des hypothèses et distincts d'une facture réelle [7].

Scénario documenté	Charge mensuelle supposée	Estimation
Hybride réduit	1 000 commandes	98,45 USD par mois
Cible hors saison	1 000 commandes	4 652,46 USD par mois
Cible en pointe	100 000 commandes	5 039,38 USD par mois

Le scénario industriel retient onze mois faibles et un mois de pointe : 11 × 4 652,46 + 5 039,38 = 56 216,44 USD par an. Le maintien du secours, des bases et des composants réseau limite la baisse du coût hors saison. Shield Advanced représente à lui seul environ 3 000 USD mensuels dans l'estimation conservée.

Ces scénarios n'offrent pas le même niveau de service et ne permettent pas de conclure qu'une architecture est intrinsèquement moins chère pour toute charge. Pour la démonstration, le socle hybride est plus proportionné. Il n'est toutefois pas gratuit sur AWS : NAT, ALB et RDS restent permanents. Les crédits disponibles et les conditions du compte doivent être vérifiés avant tout déploiement. Le budget configuré est une alerte, pas un arrêt automatique des dépenses.

Planche 2 Cible d'industrialisation proposée



Source : schéma du projet. Étude théorique non déployée.

4 Réalisation et reproductibilité

4.1 Organisation technique

Le backend utilise Python, FastAPI et SQLAlchemy. Alembic décrit les évolutions du schéma ; la seconde migration ajoute les comptes et sessions locales. Le frontend utilise HTML, CSS et JavaScript natifs. Cette structure limite les dépendances côté navigateur et laisse les règles de prix, de stock et d'accès dans l'API.

Dossier	Responsabilité
backend	API, règles métier, accès SQL, migrations et emails
frontend	Catalogue, comptes, panier et suivi des commandes
infrastructure	Ressources et tests Terraform
tests	Vérifications des comportements métier et techniques
scripts	Construction, publication et contrôle documentaire
doc et .github	Rapport, preuves, guides et validation continue

Chaque dossier source possède un README. Les versions des dépendances sont conservées dans les fichiers de dépendances et de verrouillage. Les données locales, les secrets, les plans et les états Terraform sont exclus de Git ; les fichiers exemples permettent de préparer une configuration propre à chaque compte.

4.2 Installation locale

Docker Compose est le parcours conseillé pour reproduire la démonstration avec PostgreSQL. Depuis la racine du dépôt, la commande suivante construit l'application, attend la disponibilité de la base, applique les migrations et démarre l'API et le worker :

```
docker compose up --build -d
```

La boutique est accessible sur http://localhost:8000 et la documentation API sur /api/docs. Le compte démo permet de passer une commande ; l'historique montre les trois étapes et les logs du worker simulent les emails. La commande docker compose down arrête les services tout en conservant le volume SQL.

Pour développer sans Docker, make install prépare l'environnement Python, make init initialise SQLite et make serve lance l'API. Un second terminal exécute make worker. SQLite simplifie l'installation mais ne reproduit pas les verrous PostgreSQL ; les scénarios concurrents doivent donc être vérifiés sur ce dernier moteur.

4.3 Déploiement et sécurité

La préparation AWS comprend les tests, make build, la configuration de terraform.tfvars, puis terraform init, validate, test et plan. Le plan doit être examiné avant apply. Le script deploy_frontend.py publie ensuite les fichiers sur S3 et actualise le cache CloudFront. Les archives applicatives sont construites pour Linux x86_64, indépendamment du système de développement.

Le profil et l'identifiant du compte cible peuvent être contrôlés afin d'éviter une publication dans le mauvais compte. Les secrets SQL sont lus côté serveur ; le compte applicatif a des droits limités et la migration utilise un compte distinct. Les jetons Cognito sont vérifiés dans l'API et chaque client ne peut consulter que ses commandes.

La validation SES de l'expéditeur et les restrictions du sandbox font partie de la procédure. Le déploiement est accompagné d'un guide d'exploitation pour les incidents, le rejeu et le nettoyage. Une destruction du profil de démonstration peut supprimer définitivement les données : la sauvegarde et les protections doivent être choisies avant cette opération.

4.4 Déroulement et décisions du projet

Les éléments conservés permettent de distinguer quatre étapes. Le socle local rend le parcours métier testable sans compte AWS. La mise en place de Terraform décrit ensuite le réseau, les services et les permissions. Les essais cloud exposent les écarts que les simulations ne peuvent pas reproduire. Enfin, la consolidation du dépôt rend les choix, les preuves et leurs limites accessibles au lecteur.

Étape documentée	Résultat et décision
28 octobre	Validation locale et préparation des archives. La séparation API et traitements permet de tester la logique sans AWS.
29 octobre	Essais via CloudFront, Cognito et RDS. Correction des restrictions SES puis vérification des deux emails.
29 et 30 octobre	Estimations de la solution réduite et de la cible saisonnière. Le coût permanent conduit à distinguer démonstration et exploitation industrielle.
30 octobre	Consolidation documentaire et tests locaux. Clarification de la limite Single-AZ et de la portée réelle de l'option de redondance.

4.5 Reproduire le déploiement Terraform

Prérequis : Python 3.12 ou ultérieur, Node.js 24, Terraform 1.10 ou ultérieur, AWS CLI v2 et un compte autorisant les services. Les commandes sont lancées depuis la racine du dépôt. Avant le plan, renseigner le profil, le compte cible et les emails dans le fichier de variables local.

```
make install
npm ci
make check
make build
# Créer le fichier seulement s'il n'existe pas déjà.
test -f infrastructure/terraform.tfvars || \
  cp infrastructure/terraform.tfvars.example infrastructure/terraform.tfvars
terraform -chdir=infrastructure init
terraform -chdir=infrastructure validate
terraform -chdir=infrastructure test
terraform -chdir=infrastructure plan -out=deployment.tfplan
```

Le plan montre les créations, remplacements et suppressions prévus. Après examen du compte, du périmètre et des frais, appliquer la version et publier l'interface :

```
terraform -chdir=infrastructure apply deployment.tfplan
.venv/bin/python scripts/deploy_frontend.py
terraform -chdir=infrastructure output site_url
```

Confirmer l'identité SES et les abonnements d'alerte. Dans le sandbox SES, valider également le destinataire de démonstration. Le guide doc/deploiement.md complète ces étapes. Le fichier d'état contient des données sensibles et ne doit pas être placé dans Git. La reconstruction Docker du contrôle documentaire a été interrompue lors de la récupération des métadonnées de l'image Python ; elle n'est pas comptée parmi les validations réussies de cette session.

5 Validation et résultats

5.1 Méthode

Nous distinguons les tests automatisés locaux, les plans Terraform simulés et les essais sur AWS. Cette séparation évite d'attribuer à un test un niveau de preuve qu'il ne possède pas. Les tests métier portent sur les montants, le stock, l'isolation des comptes, les reprises et les transitions. Les tests PostgreSQL créent puis suppriment des schémas temporaires isolés.

5.2 Contrôles du 30 octobre 2026

Contrôle	Résultat	Portée
Python avec PostgreSQL	90 réussis	API, transactions, comptes, événements et migrations
JavaScript	3 réussis	Calculs du panier et clé de nouvelle tentative
Ruff et Prettier	Réussis	Style et contrôles statiques configurés
Terraform validate	Réussi	Validité de la configuration
Terraform test	8 scénarios réussis	Plans avec fournisseurs simulés
Documentation	15 dossiers source documentés	Présence des README

Les quatre cas ignorés concernent des scénarios de concurrence non applicables à SQLite ; leurs variantes PostgreSQL sont exécutées. Sans configuration PostgreSQL, le même ensemble compte 56 tests réussis et 38 ignorés. Un avertissement de dépréciation Starlette/HTTPX demeure : il ne provoque pas d'échec mais devra être traité lors de l'évolution des dépendances de test.

La validation a révélé des attentes Terraform obsolètes : un test supposait encore RDS Multi-AZ avec un dimensionnement supérieur. Le code et le schéma récent prévoient au contraire une base réduite inchangée. Le test et la description de l'option ont été alignés sur ce périmètre, sans modifier les ressources. Ce résultat confirme la cohérence de la configuration documentée ; il ne valide pas une haute disponibilité SQL.

5.3 Essais AWS conservés

Le compte rendu du 29 octobre rapporte 49 vérifications réussies via CloudFront : catalogue, authentification Cognito, refus des accès anonymes, validation des paniers, idempotence, isolation entre clients et progression jusqu'à l'expédition simulée. Les comptes rendus complémentaires documentent les deux notifications et la réception d'emails dans une boîte réelle [8].

Un blocage SES associait un destinataire non vérifié en sandbox et une permission IAM trop restrictive pour le chemin d'envoi. Leur correction a permis de retraiter les événements existants sans recréer la commande. Cet exemple illustre l'intérêt du découplage asynchrone pour conserver la commande lorsque la notification échoue.

5.4 Limites des résultats

Ces essais sont datés et ne garantissent pas la disponibilité actuelle du site. La confirmation d'inscription avec un code effectivement reçu n'a pas été exercée dans le parcours initial. Les tests Terraform utilisent des fournisseurs simulés et ne prouvent pas les permissions ni les quotas du compte.

Aucun résultat de charge, de restauration complète ou de bascule régionale n'établit un niveau de disponibilité. Les preuves fonctionnelles et la réception des emails doivent donc être distinguées d'une validation de production. Le contrôle documentaire du 30 octobre n'effectue aucun nouveau déploiement AWS.

5.5 Scénario de démonstration pour l'évaluation

Démarrer l'environnement local avec Docker Compose et ouvrir http://localhost:8000. Les services appliquent les migrations avant de lancer l'API. Les emails locaux sont simulés dans les logs du worker ; leur absence dans une boîte email ne constitue pas un échec de ce mode.

Étape	Action	Résultat attendu
1	Ouvrir le catalogue	Douze produits et leurs prix sont affichés.
2	Choisir le compte démo puis ajouter un produit	Le panier contient la quantité et le montant calculés.
3	Confirmer la commande	Une seule commande est enregistrée et la confirmation s'affiche.
4	Consulter l'historique	La progression atteint l'emballage puis l'expédition simulée.
5	Consulter les logs du worker	Deux notifications simulées correspondent à la confirmation et à l'expédition.

5.6 Vérifier une nouvelle tentative et les accès

La requête locale suivante crée une commande. La rejouer sans la modifier doit renvoyer le même identifiant, sans nouvelle réservation de stock. Modifier le panier tout en gardant cette clé doit produire une réponse 409. Utiliser une clé différente pour une nouvelle commande volontaire.

```
curl -X POST http://localhost:8000/api/orders \
  -H 'Content-Type: application/json' \
  -H 'Authorization: Bearer local-demo' \
  -H 'Idempotency-Key: demonstration-0001' \
  -d '{"items":[{"product_id":"citrouille","quantity":2}]}'
```

Sans authentification, la consultation des commandes doit être refusée avec 401. La lecture de la commande d'un autre client doit renvoyer 404. L'ajout d'un prix au corps de commande doit être refusé avec 422. Les tests automatisés vérifient ces comportements sans dépendre d'une observation manuelle.

5.7 Relancer les contrôles automatisés

```
make lint
docker compose up -d db
TEST_DATABASE_URL='postgresql+psycopg://halloween:local-only-password@127.0.0.1:5432/halloween' \
  make test
terraform -chdir=infrastructure validate
terraform -chdir=infrastructure test
```

Les essais avec PostgreSQL utilisent des schémas temporaires. Sur AWS, remplacer le compte démo par une connexion Cognito, puis vérifier les événements, les emails SES et l'absence de messages en file d'échec. Ne pas utiliser les identifiants locaux pour ce parcours. Une démonstration fonctionnelle ne mesure pas la résistance à une panne ni la capacité en charge.

6 Discussion et apprentissages

6.1 Cohérence métier et systèmes distribués

La principale difficulté dépasse la création d'une route HTTP. Une commande doit rester cohérente lorsque le navigateur réessaie, lorsque deux clients achètent simultanément ou lorsqu'un consommateur reçoit plusieurs fois un message. Les transactions SQL, le contrôle du stock et l'idempotence répondent à des risques différents ; aucun de ces mécanismes ne remplace les autres.

L'outbox nous conduit à traiter explicitement les frontières entre systèmes. La base peut rendre atomiques ses propres écritures, mais pas un appel SES suivi d'une écriture SQL. Reconnaître cette limite permet de définir une reprise réaliste plutôt que de promettre une absence universelle de doublons.

6.2 Écarts entre local et cloud

Le partage du code métier entre le worker local et les Lambdas facilite la compréhension et les tests. Il ne rend pas les environnements équivalents. PostgreSQL apporte des verrous différents de SQLite ; AWS ajoute les permissions, les quotas, le réseau et les conditions d'utilisation de SES.

Les essais réels ont montré qu'une commande peut être correctement traitée alors que sa notification reste bloquée. Le suivi des files, des reçus et des erreurs permet de distinguer ces états. Cette démarche est plus utile qu'une simple vérification visuelle de la page de confirmation.

6.3 Disponibilité et proportionnalité

Une répartition des instances sur deux zones ne supprime pas le point de défaillance d'une base Single-AZ. La documentation doit décrire le périmètre réellement protégé. De même, un environnement de secours sur un schéma ne démontre ni le temps de reprise ni la capacité à éviter des effets métier répétés.

La cible industrielle améliore les possibilités d'exploitation mais augmente le socle de coût et le nombre de composants à maintenir. Avant d'ajouter ces services, il faut définir les objectifs de disponibilité, mesurer la charge et estimer le coût d'une interruption. Pour une boutique pédagogique, le maintien permanent d'un secours et d'une protection premium n'est pas proportionné au seul besoin de démonstration.

6.4 Perspectives hiérarchisées

1. Définir des critères mesurables : latence acceptable, débit de commandes, disponibilité visée, RTO et RPO.
2. Exécuter des tests de charge et de concurrence représentatifs, mesurer les connexions SQL et le retard de traitement des files.
3. Tester une restauration de base et une procédure de rejeu avant d'envisager un secours multi-région.
4. Réaliser un prototype de l'alternative entièrement serverless et comparer les coûts à niveau de service équivalent.
5. Renforcer progressivement la base et le déploiement, avec des migrations compatibles et des procédures de retour arrière testées.

Ces étapes donnent un ordre de progression : établir les besoins et les preuves avant d'augmenter la complexité. Elles permettent aussi de transformer les limites observées en objectifs de travail vérifiables.

7 Conclusion et références

7.1 Conclusion

Minuit propose une réalisation pédagogique qui relie interface web, transactions relationnelles et traitements événementiels. L'infrastructure hybride associe le contrôle d'une API sur EC2 à des consommateurs Lambda, tandis que Terraform et Docker contribuent à sa reproductibilité. Les tests vérifient les principales règles métier et les essais AWS conservés montrent un parcours de commande avec notifications.

La comparaison avec la cible industrielle montre que disponibilité, coût et complexité doivent être arbitrés ensemble. Le profil livré conserve des limites explicites, notamment RDS Single-AZ et l'absence de validation en charge ou de reprise régionale. La solution entièrement serverless reste une alternative à prototyper, distincte de la cible industrielle hybride.

Notre principal apprentissage est de relier chaque promesse à une preuve : un mécanisme transactionnel à ses tests, une disponibilité à un exercice de panne, un coût à des hypothèses de charge. Cette discipline fournit une base pour faire évoluer la démonstration vers une plateforme dont les propriétés pourraient être mesurées et exploitées.

7.2 Références techniques

Documentation officielle consultée le 30 octobre 2026.

[4] AWS Prescriptive Guidance. Transactional outbox pattern.

https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/transactional-outbox.html

[5] AWS Lambda Developer Guide. Using Lambda with Amazon SQS.

https://docs.aws.amazon.com/lambda/latest/dg/with-sqs.html

[6] Amazon CloudFront Developer Guide. Optimize high availability with CloudFront origin failover.

https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/high_availability_origin_failover.html